

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I G.Amrutha(192511250) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

amrutha

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

EXPERIMENT 1: DECISION TREE CLASSIFICATION

Aim

To implement and evaluate a Decision Tree Classifier on the Iris dataset using Python and Scikit-learn, preprocess the data, visualize the decision tree, identify the root node, and evaluate classification performance using accuracy, confusion matrix, and classification report.

Software / Tools Required

- Python 3.x
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

Theory

A Decision Tree is a supervised machine-learning algorithm that classifies data by recursively splitting it according to feature values.

Two commonly used splitting criteria are:

1. Gini Index

where p is the probability of a sample belonging to class c .

A lower Gini value indicates a purer split.

2. Information Gain

Information Gain is based on entropy:

The feature producing the largest information gain is preferred.

(a) Load, Preprocess and Split the Dataset

The Iris dataset contains 150 samples belonging to three classes:

- Setosa
- Versicolor
- Virginica

Each sample contains four numerical features:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

Python Program

```
# EXPERIMENT 1: DECISION TREE CLASSIFICATION
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
```

```
from sklearn.metrics import (
```

```
    accuracy_score,
```

```
    confusion_matrix,
```

```
    classification_report
```

```
)
```

```
# -----
```

```
# 1. Load Iris dataset
```

```
# -----
```

```

iris = load_iris()

X = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

y = pd.Series(
    iris.target,
    name="target"
)

# Convert target numbers into class names
y_names = y.map(dict(enumerate(iris.target_names)))

# Combine features and target
df = X.copy()
df["target"] = y_names

# -----
# 2. Display first 5 rows
# -----

print("FIRST 5 ROWS:")
print(df.head())

# -----
# 3. Display data types
# -----

print("\nDATA TYPES:")
print(df.dtypes)

# -----
# 4. Check missing values

```

```
# -----

print("\nMISSING VALUES:")
print(df.isnull().sum())

# -----
# 5. Handle missing values
# -----

X = X.fillna(X.median())

# -----
# 6. Split dataset
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# -----
# 7. Build Decision Tree
# -----

model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

model.fit(X_train, y_train)
```

```

# -----
# 8. Print tree structure
# -----

print("\nDECISION TREE STRUCTURE:")
print(export_text(
    model,
    feature_names=list(X.columns)
))

# -----
# 9. Identify root node
# -----

root_feature_index = model.tree_.feature[0]
root_threshold = model.tree_.threshold[0]

print("\nROOT NODE:")
print("Feature:", X.columns[root_feature_index])
print("Threshold:", root_threshold)

# -----
# 10. Prediction
# -----

y_pred = model.predict(X_test)

# -----
# 11. Accuracy
# -----

accuracy = accuracy_score(y_test, y_pred)

print("\nACCURACY:")
print(accuracy)

```

```

# -----
# 12. Confusion Matrix
# -----

cm = confusion_matrix(y_test, y_pred)

print("\nCONFUSION MATRIX:")
print(cm)

# -----
# 13. Classification Report
# -----

print("\nCLASSIFICATION REPORT:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)

# -----
# 14. Find misclassified instances
# -----

misclassified = np.where(y_test.values != y_pred)[0]

print("\n MISCLASSIFIED INSTANCES:")

for i in misclassified:
    print(
        "Index:", y_test.index[i],
        "Actual:", iris.target_names[y_test.iloc[i]],
        "Predicted:", iris.target_names[y_pred[i]]
    )

```



```

)

# -----
# 15. Plot Decision Tree
# -----

plt.figure(figsize=(16, 10))

plot_tree(
    model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True,
    rounded=True
)

plt.title("Decision Tree Classifier - Iris Dataset")
plt.show()

```

Expected First 5 Rows

The output will be similar to:

sepal length	sepal width	petal length	petal width	target
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
4.6	3.1	1.5	0.2	setosa
5.0	3.6	1.4	0.2	setosa

Data Types

sepal length (cm)	float64
sepal width (cm)	float64
petal length (cm)	float64
petal width (cm)	float64
target	object

The Iris feature values are numerical, so encoding of input features is not required. The target class names are categorical labels; Scikit-learn internally uses numerical class labels in this implementation.

Missing Values

The Iris dataset normally contains no missing values.

Nevertheless, the program explicitly checks for missing values and uses:

```
X = X.fillna(X.median())
```

This demonstrates appropriate preprocessing.

(b) Decision Tree Construction

The model uses:

```
criterion="gini"
```

Therefore, the tree selects splits that reduce the Gini impurity.

Root Node

For the standard Iris Decision Tree, the petal length feature is typically selected as the first/root split when using the full dataset and default tree-growing behavior.

The exact threshold can depend on the training split and configuration, so the program automatically prints the actual root feature and threshold:

```
root_feature_index = model.tree_.feature[0]
```

```
root_threshold = model.tree_.threshold[0]
```

Justification

The root feature is selected because it provides the most effective reduction in impurity among the available features at the first split.

For Iris classification, petal-related measurements are particularly discriminative, especially for separating Setosa from the other classes.

(c) Model Evaluation

The program calculates:

1. Accuracy

For example, if 29 out of 30 test samples are correctly classified:

or approximately:

The exact value may vary slightly depending on the train/test split and tree configuration.

2. Confusion Matrix

A confusion matrix shows:

- Actual classes in rows
- Predicted classes in columns

A typical result may look like:

```
[[10  0  0]
```

```
 [ 0 10  0]
```

[0 1 9]]

This indicates that most samples were correctly classified.

3. Classification Report

The report provides:

- Precision
- Recall
- F1-score
- Support

Definitions:

Misclassified Instances

The program automatically identifies all incorrectly classified test samples:

```
misclassified = np.where(y_test.values != y_pred)[0]
```

Example output format:

Index: 133 Actual: virginica Predicted: versicolor

Index: 119 Actual: virginica Predicted: versicolor

Important: Do not manually claim these exact instances unless they are produced by your execution, because the train/test split and tree settings determine the misclassified samples.

Performance Analysis

The Decision Tree generally performs very well on the Iris dataset because the classes are reasonably separable using the flower measurements.

- Setosa is particularly easy to classify.
- Versicolor and Virginica have some overlapping measurements.
- Therefore, errors are more likely between Versicolor and Virginica.
- A high accuracy indicates good generalization on the test set.
- The confusion matrix confirms which classes are confused.

Advantages

1. Easy to understand and interpret.
2. Requires little preprocessing.
3. Can model nonlinear relationships.
4. Provides an interpretable tree structure.

Limitation

A very deep tree can overfit the training data. Controlling parameters such as max_depth can reduce overfitting.

EXPERIMENT 1 RESULT

The Decision Tree Classifier was successfully implemented on the Iris dataset. The data was preprocessed and divided into training and testing sets. The tree structure and root node were obtained using the Gini

Index. The classifier achieved high classification accuracy, and the confusion matrix and classification report were used to analyze its performance.

EXPERIMENT 2: REINFORCEMENT LEARNING – Q-LEARNING

Aim

To implement Q-Learning for a 4×4 Grid World, learn an optimal policy through interaction with the environment, analyze Q-value evolution, and study convergence using cumulative rewards.

(a) Environment Definition

We use a 4×4 grid.

There are:

- 16 states
- 4 actions

Actions:

0 = Up

1 = Down

2 = Left

3 = Right

Example environment:

```
+---+---+---+---+
| S0 | S1 | S2 | S3 |
+---+---+---+---+
| S4 | XX | S6 | S7 |
+---+---+---+---+
| S8 | S9 | S10 | S11 |
+---+---+---+---+
| S12 | S13 | S14 | G |
+---+---+---+---+
```

Where:

- S0 = starting state
- G = goal
- XX = obstacle

Reward Structure

Situation	Reward
Reaching goal	+10

Situation	Reward
Normal movement	-1
Hitting obstacle	-5

Q-Learning Formula

The fundamental Q-Learning update is:

where:

- $Q(s, a)$ = current Q-value
- s = current state
- a = selected action
- r = reward
- s' = next state
- α = learning rate
- γ = discount factor

Hyperparameters

We use:

Learning rate $\alpha = 0.1$

Discount factor $\gamma = 0.9$

Exploration rate $\epsilon = 1.0$ initially

Minimum $\epsilon = 0.05$

Epsilon decay = 0.995

Episodes = 500

Maximum steps/episode = 100

Although the requirement says at least 100 episodes, using 500 episodes gives the agent more opportunity to learn a stable policy.

Python Program – Q-Learning

EXPERIMENT 2: Q-LEARNING GRID WORLD

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# -----
```

```
# 1. Grid World
```

```
# -----
```

```
ROWS = 4
```

```
COLS = 4
```

```
N_STATES = ROWS * COLS
```

```
N_ACTIONS = 4
```

```
# Actions
```

```
UP = 0
```

```
DOWN = 1
```

```
LEFT = 2
```

```
RIGHT = 3
```

```
actions = ["↑", "↓", "←", "→"]
```

```
start_state = 0
```

```
goal_state = 15
```

```
# Obstacle
```

```
obstacles = {5}
```

```
# -----
```

```
# 2. Hyperparameters
```

```
# -----
```

```
alpha = 0.1
```

```
gamma = 0.9
```

```
epsilon = 1.0
```

```
epsilon_min = 0.05
```

```
epsilon_decay = 0.995
```

```
episodes = 500
```

```
max_steps = 100
```

```
# -----
```

```
# 3. Initialize Q-table
```

```
# -----
```

```
Q = np.zeros((N_STATES, N_ACTIONS))
```

```
# Representative states
```

```
representative_states = [0, 10]
```

```
# Store Q-values at selected episodes
```

```
q_history = {}
```

```
# Store cumulative rewards
```

```
episode_rewards = []
```

```
# -----
```

```
# 4. State conversion functions
```

```
# -----
```

```
def state_to_position(state):
```

```
    return divmod(state, COLS)
```

```
def position_to_state(row, col):
```

```
    return row * COLS + col
```

```
# -----
```

```
# 5. Environment step
```

```
# -----
```

```
def step(state, action):
```

```
    row, col = state_to_position(state)
```

```
    new_row = row
```

```
    new_col = col
```

```

# Apply action
if action == UP:
    new_row -= 1
elif action == DOWN:
    new_row += 1
elif action == LEFT:
    new_col -= 1
elif action == RIGHT:
    new_col += 1

# Boundary condition
if (
    new_row < 0 or
    new_row >= ROWS or
    new_col < 0 or
    new_col >= COLS
):
    return state, -5, False

next_state = position_to_state(new_row, new_col)

# Obstacle
if next_state in obstacles:
    return state, -5, False

# Goal
if next_state == goal_state:
    return next_state, 10, True

# Normal movement
return next_state, -1, False

```

6. Q-Learning algorithm


```
# -----
```

```
for episode in range(1, episodes + 1):
```

```
    state = start_state
```

```
    total_reward = 0
```

```
    for step_count in range(max_steps):
```

```
        # Epsilon-greedy action selection
```

```
        if np.random.random() < epsilon:
```

```
            action = np.random.randint(N_ACTIONS)
```

```
        else:
```

```
            action = np.argmax(Q[state])
```

```
        next_state, reward, done = step(state, action)
```

```
        # Q-learning update
```

```
        Q[state, action] = Q[state, action] + alpha * (
```

```
            reward
```

```
            + gamma * np.max(Q[next_state])
```

```
            - Q[state, action]
```

```
        )
```

```
        state = next_state
```

```
        total_reward += reward
```

```
    if done:
```

```
        break
```

```
episode_rewards.append(total_reward)
```

```
# Record Q-values
```

```
if episode in [1, 50, 100]:
```

```
    q_history[episode] = Q[representative_states].copy()
```

```
# Decay epsilon
epsilon = max(
    epsilon_min,
    epsilon * epsilon_decay
)
```

```
# -----
# 7. Display Q-values
# -----
```

```
for episode in [1, 50, 100]:
```

```
    print("\nEpisode", episode)
```

```
    for i, state in enumerate(representative_states):
```

```
        print(
            "State", state,
            "Q-values:",
            np.round(q_history[episode][i], 3)
        )
```

```
# -----
# 8. Display final Q-table
# -----
```

```
print("\nFINAL Q-TABLE:")
```

```
print(np.round(Q, 3))
```

```
# -----
# 9. Extract final policy
# -----
```

```

print("\nFINAL LEARNED POLICY:")

for row in range(ROWS):

    policy_row = []

    for col in range(COLS):

        state = position_to_state(row, col)

        if state == goal_state:
            policy_row.append("G")

        elif state in obstacles:
            policy_row.append("X")

        else:
            best_action = np.argmax(Q[state])
            policy_row.append(actions[best_action])

    print(policy_row)

# -----
# 10. Plot cumulative reward
# -----

plt.figure(figsize=(10, 5))

plt.plot(
    episode_rewards,
    label="Reward per Episode"
)

plt.xlabel("Episode")

```

```

plt.ylabel("Cumulative Reward")
plt.title("Q-Learning Convergence")
plt.grid(True)
plt.legend()

plt.show()

# -----
# 11. Moving average
# -----

window = 20

moving_average = np.convolve(
    episode_rewards,
    np.ones(window) / window,
    mode="valid"
)

plt.figure(figsize=(10, 5))

plt.plot(moving_average)

plt.xlabel("Episode")
plt.ylabel("Average Cumulative Reward")
plt.title("Moving Average Reward")

plt.grid(True)

plt.show()

```

(b) Q-Table Evolution

Initially:

Q-table = 0

because there is no prior knowledge.

For example, for representative states 0 and 10:

Episode 1

State 0:

[0.0, 0.0, 0.0, 0.0]

State 10:

[0.0, 0.0, 0.0, 0.0]

The first episode begins with no learned information.

After repeated interaction, non-zero Q-values appear.

Episode 50

The output may resemble:

State 0:

[-0.5, -0.3, -0.5, 1.2]

State 10:

[-0.4, 4.8, -0.5, 5.6]

Episode 100

The Q-values may resemble:

State 0:

[-0.6, 1.8, -0.6, 4.5]

State 10:

[2.1, 7.2, 1.4, 8.1]

These numerical values are illustrative. Because Q-Learning uses random exploration, your exact values will differ between runs unless you fix a random seed.

The important observation is:

with useful actions gradually acquiring higher Q-values.

How the Q-Table Evolves

Initial stage

All Q-values = 0

The agent has no knowledge about the environment.

Learning stage

The agent explores different actions.

Rewards propagate backward through the Q-table.

Actions leading toward the goal gradually receive higher Q-values.

Convergence stage

The Q-values become relatively stable.

The best action can be selected using:

`np.argmax(Q[state])`

Thus, the Q-table represents the learned policy.

(c) Final Learned Policy

The policy is extracted by selecting the action having the maximum Q-value:

Example final policy:

→ → → ↓

↓ X ↓ ↓

→ → → ↓

→ → → G

Here:

- ↑ = Up
- ↓ = Down
- ← = Left
- → = Right
- X = obstacle
- G = goal

The exact learned policy may vary because of exploration and the randomly initialized action selection.

Reward Graph Analysis

The program plots:

`plt.plot(episode_rewards)`

Initially, the cumulative reward can be low because the agent:

- explores random actions,
- hits boundaries,
- encounters the obstacle,
- takes longer paths.

As learning progresses:

- useful paths acquire higher Q-values,
- exploration decreases,
- the agent reaches the goal more consistently,
- cumulative reward generally improves.

The moving-average plot makes this trend easier to observe.

Convergence Justification

Q-Learning gradually converges because the agent repeatedly updates its estimates according to:

At the beginning, the Q-values change significantly because the agent has little knowledge.

After many episodes, the agent repeatedly experiences similar state-action transitions. Consequently, Q-values become more stable.

The decreasing value of ϵ also reduces random exploration:

$$\epsilon = 1.0 \rightarrow 0.05$$

Therefore, the agent increasingly exploits the learned best actions.

However, convergence is not guaranteed to look perfectly smooth in every episode, because exploration and stochastic behavior can still cause occasional low-reward episodes.

Important Q-Learning Parameters

Parameter	Value	Purpose
α	0.1	Learning rate
γ	0.9	Importance of future rewards
ϵ	1.0	Initial exploration
ϵ minimum	0.05	Minimum exploration
ϵ decay	0.995	Gradually reduces exploration
Episodes	500	Training iterations
Max steps	100	Prevents endless episodes

Comparison of the Two Experiments

Aspect	Decision Tree	Q-Learning
Learning type	Supervised	Reinforcement
Input	Labeled dataset	Environment interaction
Learning signal	Correct class labels	Rewards
Main structure	Tree	Q-table
Objective	Classification	Optimal policy
Exploration	Not required	Required
Output	Predicted class	Best action

Aspect	Decision Tree	Q-Learning
Evaluation	Accuracy, confusion matrix, F1	Reward and convergence
Example	Iris classification	Grid World

Rubric-Based Analysis

1. Understanding of Concepts – 25%

Both experiments demonstrate different machine-learning paradigms.

Decision Tree: learns classification rules from labeled examples.

Q-Learning: learns state-action values through trial and error using rewards.

The experiments also demonstrate important concepts such as Gini impurity, entropy, Q-values, exploration, exploitation, learning rate, and discount factor.

2. Experimental Design – 30%

The experiments use systematic procedures:

Decision Tree

Load Dataset



Preprocessing



Train/Test Split



Train Decision Tree



Prediction



Evaluation

Q-Learning

Initialize Environment



Initialize Q-table



Select Action



Receive Reward



Update Q-value



Repeat Episodes



Extract Policy

3. Use of Tools – 20%

Python and Scikit-learn are used for the Decision Tree experiment.

NumPy and Matplotlib are used for Q-Learning and reward visualization.

Important libraries:

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.tree import DecisionTreeClassifier
```

4. Analysis of Results – 15%

The Decision Tree should achieve strong performance on Iris because its classes can be effectively separated using flower measurements.

Q-Learning should progressively learn a route toward the goal. The Q-values become more meaningful over episodes, while the reward trend generally improves as the policy becomes better.

5. Documentation – 10%

For your laboratory record, include:

1. Aim
2. Software requirements
3. Theory
4. Algorithm
5. Python program
6. Dataset/environment description
7. Output
8. Decision-tree diagram
9. Confusion matrix
10. Q-table snapshots
11. Policy diagram
12. Reward graph
13. Result
14. Conclusion

FINAL CONCLUSION

Experiment 1

The Decision Tree Classifier was successfully implemented using the Iris dataset. The dataset was checked for missing values, split into training and testing sets, and classified using the Gini Index. Accuracy, confusion matrix, precision, recall, and F1-score were used to evaluate the classifier. The tree structure also provided an interpretable explanation of the classification decisions.

Experiment 2

The Q-Learning algorithm was successfully implemented in a 4×4 Grid World. Initially, the Q-table contained zeros, but repeated interaction with the environment allowed the agent to learn useful state-action values. The final policy selected actions with the highest Q-values and guided the agent toward the goal while avoiding the obstacle. The cumulative-reward graph demonstrated the learning and convergence behaviour.

Overall, these experiments demonstrate the difference between supervised learning through Decision Trees and reinforcement learning through Q-Learning.